

9 Bayesian methods

This section is dedicated to the subset of machine learning that makes prior assumptions on parameters. Before we explain how Bayes' theorem can be applied to simple building blocks in machine learning, we introduce some notations and concepts in the subsection below. Good references for Bayesian analysis are Gelman et al. (2013) and Kruschke (2014). The latter, like the present book, illustrates the concepts with many lines of R code. Bayesian inference is used in Feng and He (2021) to estimate conditional expected returns and residual covariance matrices with a view to portfolio choice.

9.1 The Bayesian framework

Up to now, the models that have been presented rely on data only. This approach is often referred to as '**frequentist**'. Given one dataset, a frequentist will extract (i.e., estimate) a unique set of optimal parameters and consider it to be the best model. Bayesians, on the other hand, consider datasets as **snapshots of reality** and, for them, parameters are thus random! Instead of estimating one value for parameters (e.g., a coefficient in a linear model), they are more ambitious and try to determine the **whole distribution** of the parameter.

In order to outline how that can be achieved, we introduce basic notations and results. The foundational concept in Bayesian analysis is the **conditional probability**. Given two random sets (or events) A and B , we define the probability of A knowing B (equivalently, the odds of having A , conditionally on having B) as

$$P[A|B] = \frac{P[A \cap B]}{P[B]},$$

that is, the probability of the intersection between the two sets divided by the probability of B . Likewise, the probability that both events occur is equal to $P[A \cap B] = P[A]P[B|A]$. Given n disjoint events A_i , $i = 1, \dots, n$ such that $\sum_{i=1}^n P(A_i) = 1$, then for any event B , the law of total probabilities is (or implies)

$$P(B) = \sum_{i=1}^n P(B \cap A_i) = \sum_{i=1}^n P(B|A_i)P(A_i).$$

Given this expression, we can formulate a general version of Bayes' theorem:

$$P(A_i|B) = \frac{P(A_i)P(B|A_i)}{P(B)} = \frac{P(A_i)P(B|A_i)}{\sum_{i=1}^n P(B|A_i)P(A_i)}. \quad (9.1)$$

Endowed with this result, we can move forward to the core topic of this section, which is the estimation of some parameter θ (possibly a vector) given a dataset, which we denote with $\mathbf{y}$ thereby following the conventions from Gelman et al. (2013). This notation is suboptimal in this book nonetheless because in all other chapters, $\mathbf{y}$ stands for the label of a dataset.

In Bayesian analysis, one sophistication (compared to a frequentist approach) comes from the fact that the data is not almighty. The distribution of the parameter θ will be a mix between some **prior** distribution set by the statistician (the user, the analyst) and the empirical distribution from the data. More precisely, a simple application of Bayes' formula yields

$$p(\theta|\mathbf{y}) = \frac{p(\theta)p(\mathbf{y}|\theta)}{p(\mathbf{y})} \propto p(\theta)p(\mathbf{y}|\theta). \quad (9.2)$$

The interpretation is immediate: the distribution of θ knowing the data $\mathbf{y}$ is proportional to the distribution of θ times the distribution of $\mathbf{y}$ knowing θ . The term $p(\mathbf{y})$ is often omitted because it is simply a scaling number that ensures that the density sums or integrates to one.

We use a slightly different notation between Equation (9.1) and Equation (9.2). In the former, P denotes a true probability, i.e., it is a number. In the latter, p stands for the whole probability density function of θ or $\mathbf{y}$.

The whole purpose of Bayesian analysis is to compute the so-called **posterior** distribution $p(\theta|\mathbf{y})$ via the **prior** distribution $p(\theta)$ and the **likelihood function** $p(\mathbf{y}|\theta)$. Priors are sometimes qualified as informative, weakly informative or uninformative, depending on the degree to which the user is confident on the relevance and robustness of the prior. The simplest way to define a non-informative prior is to set a constant (uniform) distribution over some realistic interval(s).

The most challenging part is usually the likelihood function. The easiest way to solve the problem is to resort to a specific distribution (possibly a parametric family) for the distribution of the data and then consider that observations are i.i.d., just as in a simple maximum likelihood inference. If we assume that new parameters for the distributions are gathered into $\boldsymbol{\lambda}$, then the likelihood can be written as

$$p(\mathbf{y}|\boldsymbol{\theta}, \boldsymbol{\lambda}) = \prod_{i=1}^I f_{\lambda}(y_i; \boldsymbol{\beta}), \quad (9.3)$$

but in this case the problem becomes slightly more complex because adding new parameters changes the posterior distribution to $p(\boldsymbol{\theta}, \boldsymbol{\lambda}|\mathbf{y})$. The user must find out the joint distribution of $\boldsymbol{\theta}$ and $\boldsymbol{\lambda}$ - given $\mathbf{y}$. Because of their nested structure, these models are often called **hierarchical models**.

Bayesian methods are widely used for portfolio choice. The rationale is that the distribution of asset returns depends on some parameter and the main issue is to determine the posterior distribution. We very briefly review a vast literature below. Bayesian asset allocation is investigated in Lai et al. (2011) (via stochastic optimization), Guidolin and Liu (2016) and Dangl and Weissensteiner (2020). Shrinkage techniques (of means and covariance matrices) are tested in Frost and Savarino (1986), Kan and Zhou (2007) and DeMiguel, Martín-Utrera, and Nogales (2015). In a similar vein, Tu and Zhou (2010) build priors that are coherent with asset pricing theories. Finally, Bauder et al. (2020) sample portfolio returns which allows to derive a Bayesian optimal frontier. We invite the interested reader to also dwell in the references that are cited within these few articles.

9.2 Bayesian sampling

9.2.1 Gibbs sampling

One adjacent field of applications of Bayes' theorem is **simulation**. Suppose we want to simulate the multivariate distribution of a random vector $\mathbf{X}$ given by its density $p = p(x_1, \dots, x_J)$. Often, the full distribution is complex, but its marginals are more accessible. Indeed, they are simpler because they depend on only one variable (when all other values are known):

$$p(X_j = x_j | X_1 = x_1, \dots, X_{j-1} = x_{j-1}, X_{j+1} = x_{j+1}, \dots, X_J = x_J) = p(X_j = x_j | \mathbf{X}_{-j} = \mathbf{x}_{-j}),$$

where we use the compact notation $\mathbf{X}_{-j}$ for all variables except X_j . One way to generate samples with law p is the following and relies both on the knowledge of the conditionals $p(x_j | \mathbf{x}_{-j})$ and on the notion of **Markov Chain Monte Carlo**, which we outline below. The process is iterative and assumes that it is possible to draw samples of the aforementioned conditionals. We write x_j^m for the m^{th} sample of the j^{th} variable (X_j). The simulation starts with a prior (or fixed, or random) sample $\mathbf{x}^0 = (x_1^0, \dots, x_J^0)$. Then, for a sufficiently large number of times, say T , new samples are drawn according to

$$\begin{aligned} x_1^{m+1} &= p(X_1 | X_2 = x_2^m, \dots, X_J = x_J^m); \\ x_2^{m+1} &= p(X_2 | X_1 = x_1^{m+1}, X_3 = x_3^m, \dots, X_J = x_J^m); \\ &\dots \\ x_J^{m+1} &= p(X_J | X_1 = x_1^{m+1}, X_2 = x_2^{m+1}, \dots, X_{J-1} = x_{J-1}^{m+1}). \end{aligned}$$

The important detail is that after each line, the value of the variable is updated. Hence, in the second line, X_2 is sampled with the knowledge of $X_1 = x_1^{m+1}$ and in the last line, all variables except X_J have been updated to their $(m+1)^{th}$ state. The above algorithm is called Gibbs sampling. It relates to Markov chains because each new iteration depends only on the previous one.

Under some technical assumptions, as T increases, the distribution of $\mathbf{x}_T$ converges to that of p . The conditions under which the convergence occurs have been widely discussed in series of articles in the 1990s. The interested reader can have a look for instance at Tierney (1994), Roberts and Smith (1994), as well as at section 11.7 of Gelman et al. (2013).

Sometimes, the full distribution is complex and the conditional laws are hard to determine and to sample. Then, a more general method, called Metropolis-Hastings, can be used that relies on the rejection method for the simulation of random variables.

9.2.2 Metropolis-Hastings sampling

The Gibbs algorithm can be considered as a particular case of the Metropolis-Hastings (MH) method, which, in its simplest version, was introduced in Metropolis and Ulam (1949). The premise is similar: the aim is to simulate random variables that follow $p(\mathbf{x})$ with the ability to sample from a simpler form $p(\mathbf{x}|\mathbf{y})$ which gives the probability of the future state $\mathbf{x}$, given the past one $\mathbf{y}$.

Once an initial value for $\mathbf{x}$ has been sampled ($\mathbf{x}_0$), each new iteration (m) of the simulation takes place in three stages:

1. generate a candidate value $\mathbf{x}'_{m+1}$ from $p(\mathbf{x}|\mathbf{x}_m)$,
2. compute the acceptance ratio $\alpha = \min \left(\frac{p(\mathbf{x}'_{m+1})p(\mathbf{x}_m|\mathbf{x}'_{m+1})}{p(\mathbf{x}_m)p(\mathbf{x}'_{m+1}|\mathbf{x}_m)} \right)$

3. pick $\mathbf{x}_{m+1} = \mathbf{x}'_{m+1}$ with probability α or stick with the previous value ($\mathbf{x}_{m+1} = \mathbf{x}_m$) with probability $1 - \alpha$.

The interpretation of the acceptance ratio is not straightforward in the general case. When the sampling generator is symmetric ($p(\mathbf{x}|\mathbf{y}) = p(\mathbf{y}|\mathbf{x})$), the candidate is always chosen whenever $p(\mathbf{x}'_{m+1}) \geq p(\mathbf{x}_m)$. If the reverse condition holds ($p(\mathbf{x}'_{m+1}) < p(\mathbf{x}_m)$), then the candidate is retained with odds equal to $p(\mathbf{x}'_{m+1})/p(\mathbf{x}_m)$, which is the ratio of likelihoods. The more likely the new proposal, the higher the odds of retaining it.

Often, the first simulations are discarded in order to leave time to the chain to converge to a high probability region. This procedure (often called '*burn in*') ensures that the first retained samples are located in a zone that is likely, i.e., that they are more representative of the law we are trying to simulate.

For the sake of brevity, we stick to a succinct presentation here, but some additional details are outlined in section 11.2 of Gelman et al. (2013) and in chapter 7 of Kruschke (2014).

9.3 Bayesian linear regression

Because Bayesian concepts are rather abstract, it is useful to illustrate the theoretical notions with a simple example. In a linear model, $y_i = \mathbf{x}_i \mathbf{b} + \epsilon_i$ and it is often statistically assumed that the ϵ_i are i.i.d. and normally distributed with zero mean and variance σ^2 . Hence, the likelihood of Equation (9.3) translates into

$$p(\boldsymbol{\epsilon}|\mathbf{b}, \sigma) = \prod_{i=1}^I \frac{e^{-\frac{\epsilon_i^2}{2\sigma^2}}}{\sigma\sqrt{2\pi}} = (\sigma\sqrt{2\pi})^{-I} e^{-\sum_{i=1}^I \frac{\epsilon_i^2}{2\sigma^2}}.$$

In a regression analysis, the data is given both by $\mathbf{y}$ and by $\mathbf{X}$, hence both are reported in the notations. Simply acknowledging that $\boldsymbol{\epsilon} = \mathbf{y} - \mathbf{X}\mathbf{b}$, we get

$$\begin{aligned} p(\mathbf{y}, \mathbf{X}|\mathbf{b}, \sigma) &= \prod_{i=1}^I \frac{e^{-\frac{\epsilon_i^2}{2\sigma^2}}}{\sigma\sqrt{2\pi}} \\ &= (\sigma\sqrt{2\pi})^{-I} e^{-\sum_{i=1}^I \frac{(y_i - \mathbf{x}_i' \mathbf{b})^2}{2\sigma^2}} = (\sigma\sqrt{2\pi})^{-I} e^{-\frac{(\mathbf{y} - \mathbf{X}\mathbf{b})'(\mathbf{y} - \mathbf{X}\mathbf{b})}{2\sigma^2}} \\ &= \underbrace{(\sigma\sqrt{2\pi})^{-I} e^{-\frac{(\mathbf{y} - \mathbf{X}\mathbf{b})'(\mathbf{y} - \mathbf{X}\mathbf{b})}{2\sigma^2}}}_{\text{depends on } \sigma, \text{ not } \mathbf{b}} \times \underbrace{e^{-\frac{(\mathbf{b} - \hat{\mathbf{b}})' \mathbf{X}' \mathbf{X} (\mathbf{b} - \hat{\mathbf{b}})}{2\sigma^2}}}_{\text{depends on both } \sigma, \text{ and } \mathbf{b}}. \end{aligned} \quad (9.4)$$

In the last line, the second term is a function of the difference $\mathbf{b} - \hat{\mathbf{b}}$, where $\hat{\mathbf{b}} = (\mathbf{X}'\mathbf{X})^{-1}\mathbf{X}'\mathbf{y}$. This is not surprising: $\hat{\mathbf{b}}$ is a natural benchmark for the mean of $\mathbf{b}$. Moreover, introducing $\hat{\mathbf{b}}$ yields a relatively simple form for the probability.

The above expression is the frequentist (data-based) block of the posterior: the likelihood. If we want to obtain a tractable expression for the posterior, we need to find a prior component that has a form that will combine well with this likelihood. These forms are called **conjugate priors**. A natural candidate for the right part (that depends on both $\mathbf{b}$ and σ) is the multivariate Gaussian density:

$$p[\mathbf{b}|\sigma] = \sigma^{-k} e^{-\frac{(\mathbf{b} - \mathbf{b}_0)' \boldsymbol{\Lambda}_0 (\mathbf{b} - \mathbf{b}_0)}{2\sigma^2}}, \quad (9.5)$$

where we are obliged to condition with respect to σ . The density has prior mean $\mathbf{b}_0$ and prior covariance matrix $\boldsymbol{\Lambda}_0^{-1}$. This prior gets us one step closer to the posterior because

$$\begin{aligned} p[\mathbf{b}, \sigma|\mathbf{y}, \mathbf{X}] &\propto p[\mathbf{y}, \mathbf{X}|\mathbf{b}, \sigma] p[\mathbf{b}, \sigma] \\ &\propto p[\mathbf{y}, \mathbf{X}|\mathbf{b}, \sigma] p[\mathbf{b}|\sigma] p[\sigma]. \end{aligned} \quad (9.6)$$

In order to fully specify the cascade of probabilities, we need to take care of σ and set a density of the form

$$p[\sigma^2] \propto (\sigma^2)^{-1-a_0} e^{-\frac{b_0}{2\sigma^2}}, \quad (9.7)$$

which is close to that of the left part of (9.4). This corresponds to an inverse gamma distribution for the variance with prior parameters a_0 and b_0 (this scalar notation is not optimal because it can be confused with the prior mean $\mathbf{b}_0$ so we must pay extra attention).

Now, we can simplify $p[\mathbf{b}, \sigma | \mathbf{y}, \mathbf{X}]$ with (9.4), (9.5) and (9.7):

$$p[\mathbf{b}, \sigma | \mathbf{y}, \mathbf{X}] \propto (\sigma \sqrt{2\pi})^{-I} \sigma^{-2(1+a_0)} e^{-\frac{(\mathbf{y}-\mathbf{X}\hat{\mathbf{b}})'(\mathbf{y}-\mathbf{X}\hat{\mathbf{b}})}{2\sigma^2}} \\ \times e^{-\frac{(\mathbf{b}-\hat{\mathbf{b}})'\mathbf{X}'\mathbf{X}(\mathbf{b}-\hat{\mathbf{b}})}{2\sigma^2}} \sigma^{-k} e^{-\frac{(\mathbf{b}-\mathbf{b}_0)'\mathbf{\Lambda}_0(\mathbf{b}-\mathbf{b}_0)}{2\sigma^2}} e^{-\frac{b_0}{2\sigma^2}}$$

which can be rewritten

$$p[\mathbf{b}, \sigma | \mathbf{y}, \mathbf{X}] \propto \sigma^{-I-k-2(1+a_0)} \\ \times \exp\left(-\frac{(\mathbf{y}-\mathbf{X}\hat{\mathbf{b}})'(\mathbf{y}-\mathbf{X}\hat{\mathbf{b}}) + (\mathbf{b}-\hat{\mathbf{b}})'\mathbf{X}'\mathbf{X}(\mathbf{b}-\hat{\mathbf{b}}) + (\mathbf{b}-\mathbf{b}_0)'\mathbf{\Lambda}_0(\mathbf{b}-\mathbf{b}_0) + b_0}{2\sigma^2}\right).$$

The above expression is simply a quadratic form in $\mathbf{b}$ and it can be rewritten after burdensome algebra in a much more compact manner:

$$p(\mathbf{b} | \mathbf{y}, \mathbf{X}, \sigma) \propto \left[\sigma^{-k} e^{-\frac{(\mathbf{b}-\mathbf{b}_*)'\mathbf{\Lambda}_*(\mathbf{b}-\mathbf{b}_*)}{2\sigma^2}} \right] \times \left[(\sigma^2)^{-1-a_*} e^{-\frac{b_*}{2\sigma^2}} \right],$$

where

$$\mathbf{\Lambda}_* = \mathbf{X}'\mathbf{X} + \mathbf{\Lambda}_0 \\ \mathbf{b}_* = \mathbf{\Lambda}_*^{-1}(\mathbf{\Lambda}_0\mathbf{b}_0 + \mathbf{X}'\mathbf{X}\hat{\mathbf{b}}) \\ a_* = a_0 + I/2 \\ b_* = b_0 + \frac{1}{2}(\mathbf{y}'\mathbf{y} + \mathbf{b}_0'\mathbf{\Lambda}_0\mathbf{b}_0 + \mathbf{b}_*'\mathbf{\Lambda}_*\mathbf{b}_*).$$

This expression has two parts: the Gaussian component which relates mostly to $\mathbf{b}$, and the inverse gamma component, entirely dedicated to σ . The mix between the prior and the data is clear. The posterior covariance matrix of the Gaussian part ($\mathbf{\Lambda}_*$) is the sum between the prior and a quadratic form from the data. The posterior mean $\mathbf{b}_*$ is a weighted average of the prior $\mathbf{b}_0$ and the sample estimator $\hat{\mathbf{b}}$. Such blends of quantities estimated from data and a user-supplied version are often called **shrinkages**. For instance, the original matrix of cross-terms $\mathbf{X}'\mathbf{X}$ is shrunk towards the prior $\mathbf{\Lambda}_0$. This can be viewed as a **regularization** procedure: the pure fit originating from the data is mixed with some ‘external’ ingredient to give some structure to the final estimation.

The interested reader can also have a look at section 16.3 of Greene (2018) (the case of conjugate priors is treated in subsection 16.3.2).

The formulae above can be long and risky to implement. Luckily, there is an R package (*spBayes*) that performs Bayesian inference for linear regression using the conjugate priors. Below, we provide one example of how it works. To simplify the code and curtail computation times, we consider two predictors: market capitalization (size anomaly) and price-to-book ratio (value anomaly). In statistics, the **precision matrix** is the inverse of the covariance matrix. In the parameters, the first two priors relate to the Gaussian law and the last two to the inverse gamma distribution:

$$f_{\text{invgamma}}(x, \alpha, \beta) = \frac{\beta^\alpha}{\Gamma(\alpha)} x^{-1-\alpha} e^{-\frac{\beta}{x}},$$

where α is the shape and β is the scale.

```
prior_mean <- c(0.01,0.1,0.1) # Average value of parameters (prior)
precision_mat <- diag(prior_mean^2) %>% solve() # Inverse cov matrix of parameters (prior)
fit_lmBayes <- bayesLMConjugate(
  R1M_Usd ~ Mkt_Cap_3M_Usd + Pb, # Model: size and value
  data = testing_sample, # Data source, here, the test sample
  n.samples = 2000, # Number of samples used
  beta.prior.mean = prior_mean, # Avg prior: size & value rewarded & unit beta
  beta.prior.precision = precision_mat, # Precision matrix
  prior.shape = 0.5, # Shape for prior distribution of sigma
  prior.rate = 0.5) # Scale for prior distribution of sigma
```

In the above specification, we must also provide a prior for the constant. By default, we set its average value to 0.01, which corresponds to a 1% average monthly return. Once the model has been estimated, we can plot the distribution of coefficient estimates.

```
fit_lmBayes$p.beta.tauSq.samples[,1:3] %>% as_tibble() %>%
  `colnames<-`(c("Intercept", "Size", "Value")) %>%
  gather(key = coefficient, value = value) %>%
  ggplot(aes(x = value, fill = coefficient)) + geom_histogram(alpha = 0.5)
```

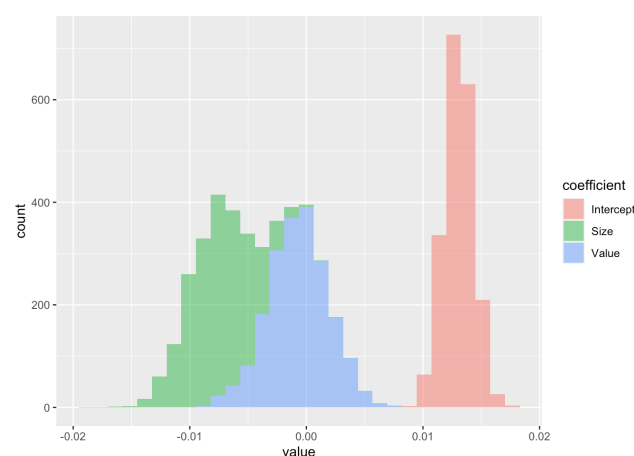


FIGURE 9.1: Distribution of linear regression coefficients (betas).

The distribution of the constant in Figure 9.1 is firmly to the right with a small dispersion, hence it is solidly positive. For the size coefficient, it is the opposite; it is negative (small firms are more profitable). With regard to value, it is hard to conclude, the distribution is balanced around zero: there is no clear exposition to the price-to-book ratio variable.

9.4 Naive Bayes classifier

Bayes' theorem can also be easily applied to **classification**. We formulate it with respect to the label and features and write

$$P[\mathbf{y}|\mathbf{X}] = \frac{P[\mathbf{X}|\mathbf{y}]P[\mathbf{y}]}{P[\mathbf{X}]} \propto P[\mathbf{X}|\mathbf{y}]P[\mathbf{y}], \quad (9.8)$$

and then split the input matrix into its column vectors $\mathbf{X} = (\mathbf{x}_1, \dots, \mathbf{x}_K)$. This yields

$$P[\mathbf{y}|\mathbf{x}_1, \dots, \mathbf{x}_K] \propto P[\mathbf{x}_1, \dots, \mathbf{x}_K|\mathbf{y}]P[\mathbf{y}]. \quad (9.9)$$

The 'naive' qualification of the method comes from a simplifying assumption on the features.¹⁹ If they are all mutually independent, then the likelihood in the above expression can be expanded into

$$P[\mathbf{y}|\mathbf{x}_1, \dots, \mathbf{x}_K] \propto P[\mathbf{y}] \prod_{k=1}^K P[\mathbf{x}_k|\mathbf{y}]. \quad (9.10)$$

The next step is to be more specific about the likelihood. This can be done non-parametrically (via kernel estimation) or with common distributions (Gaussian for continuous data, Bernoulli for binary data). In factor investing, the features are continuous, thus the Gaussian law is more adequate:

$$P[x_{i,k} = z | y_i = c] = \frac{e^{-\frac{(z-m_c)^2}{2\sigma_c^2}}}{\sigma_c \sqrt{2\pi}},$$

where c is the value of the classes taken by y and σ_c and m_c are the standard error and mean of $x_{i,k}$, conditional on y_i being equal to c . In practice, each class is spanned, the training set is filtered accordingly and σ_c and m_c are taken to be the sample statistics. This Gaussian parametrization is probably ill-suited to our dataset because the features are uniformly distributed. Even after conditioning, it is unlikely that the distribution will be even remotely close to Gaussian. Technically, this can be overcome via a double transformation method. Given a vector of features $\mathbf{x}_k$ with empirical cdf $F_{\mathbf{x}_k}$, the variable

$$\tilde{\mathbf{x}}_k = \Phi^{-1}(F_{\mathbf{x}_k}(\mathbf{x}_k)), \quad (9.11)$$

will have a standard normal law whenever $F_{\mathbf{x}_k}$ is not pathological. Non-pathological cases are when the cdf is continuous and strictly increasing and when observations lie in the open interval (0,1). If all features are independent, the transformation should not have any impact on the correlation structure. Otherwise, we refer to the literature on the NORmal-To-Anything (NORTA) method (see, e.g., H. Chen (2001) and Coqueret (2017)).

Lastly, the prior $P[\mathbf{y}]$ in Equation (9.10) is often either taken to be uniform across the classes ($1/K$ for all k) or equal to the sample distribution.

We illustrate the naive Bayes classification tool with a simple example. While the package *e1071* embeds such a classifier, the *naivebayes* library offers more options (Gaussian, Bernoulli, multinomial and nonparametric likelihoods). Below, since the features are uniformly distributed, thus the transformation in (9.11) amounts to apply the Gaussian quantile function (inverse cdf).

For visual clarity, we only use the small set of features.

```
library(naivebayes) # Load package
gauss_features_train <- training_sample %>% # Build sample
  dplyr::select(features_short) %>%
  as.matrix() %>%
  `*`(0.999) %>% # Features smaller than 1
  + (0.0001) %>% # Features larger than 0
  qnorm() %>% # Inverse Gaussian cdf
  `colnames<-`(features_short)
fit_NB_gauss <- naive_bayes(x = gauss_features_train, # Transformed features
  y = training_sample$R1M_Usd_C) # Label
layout(matrix(c(1,1,2,3,4,5,6,7), 4, 2, byrow = TRUE), # Organize graphs
  widths=c(0.9,0.45))
par(mar=c(1, 1, 1, 1))
plot(fit_NB_gauss, prob = "conditional")
```

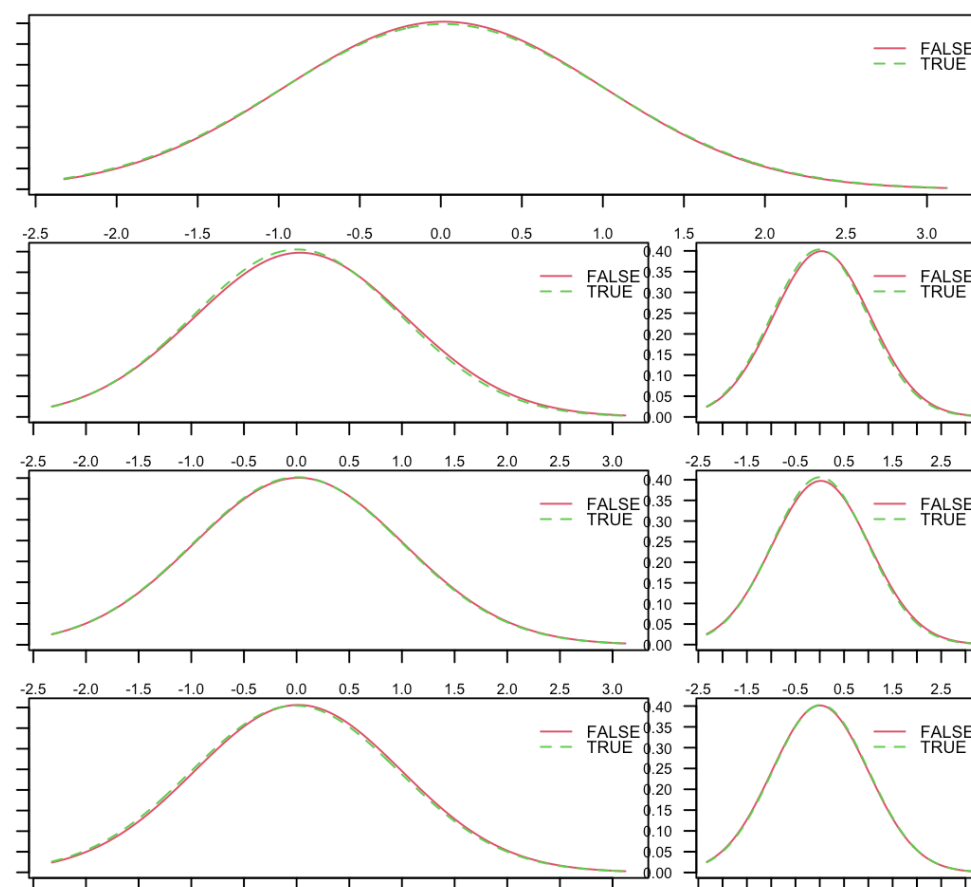


FIGURE 9.2: Distributions of predictor variables, conditional on the class of the label. TRUE is when the instance corresponds to an above median return and FALSE to a below median return.

The plots in Figure 9.2 show the distributions of the features, conditionally on each value of the label. Essentially, those are the densities $P[\mathbf{x}_k|\mathbf{y}]$. For each feature, both distributions are very similar.

As usual, once the model has been trained, the accuracy of predictions can be evaluated.

```
gauss_features_test <- testing_sample %>%
  dplyr::select(features_short) %>%
  as.matrix() %>%
  `*`(0.999) %>%
  + (0.0001) %>%
  qnorm() %>%
  `colnames<-`(features_short)
mean(predict(fit_NB_gauss, gauss_features_test) == testing_sample$R1M_Usd_C) # Hit ratio
```

The performance of the classifier is not satisfactory as it underperforms a random guess.

9.5 Bayesian additive trees

9.5.1 General formulation

Bayesian additive regression trees (BARTs) are an ensemble technique that mixes Bayesian thinking and regression trees. In spirit, they are close to the tree ensembles seen in Chapter 6, but they differ greatly in their implementation. In BARTs like in Bayesian regressions, the regularization comes from the prior. The original article is Chipman, George, and McCulloch (2010) and the implementation (in R) follows Sparapani, Spanbauer, and McCulloch (2019). BARTs have been used in Shu and Tiwari (2021) to identify which characteristics are priced. The authors report that stocks' market capitalization is the only one that matters.

Formally, the model is an aggregation of M models, which we write as

$$y = \sum_{m=1}^M \mathcal{T}_m(q_m, \mathbf{w}_m, \mathbf{x}) + \epsilon, \quad (9.12)$$

where ϵ is a Gaussian noise with variance σ^2 , and the $\mathcal{T}_m = \mathcal{T}_m(q_m, \mathbf{w}_m, \mathbf{x})$ are decision trees with structure q_m and weights vectors $\mathbf{w}_m$. This decomposition of the tree is the one we used for boosted trees and is illustrated in Figure 6.5. q_m codes all splits (variables chosen for the splits and levels of the splits) and the vectors $\mathbf{w}_m$ correspond to the leaf values (at the terminal nodes).

At the macro-level, BARTs can be viewed as traditional Bayesian objects, where the parameters θ are all of the unknowns coded through q_m , $\mathbf{w}_m$ and σ^2 and where the focus is set on determining the posterior

$$(q_m, \mathbf{w}_m, \sigma^2) | (\mathbf{X}, \mathbf{Y}). \quad (9.13)$$

Given particular forms of priors for $(q_m, \mathbf{w}_m, \sigma^2)$, the algorithm draws the parameters using a combination of Metropolis-Hastings and Gibbs samplers.

9.5.2 Priors

The definition of priors in tree models is delicate and intricate. The first important assumption is independence: independence between σ^2 and all other parameters and independence between trees, that is, between couples $(q_m, \mathbf{w}_m)$ and $(q_n, \mathbf{w}_n)$ for $m \neq n$. This assumption makes BARTs closer to random forests in spirit and further from boosted trees. This independence entails

$$P((q_1, \mathbf{w}_1), \dots, (q_M, \mathbf{w}_M), \sigma^2) = P(\sigma^2) \prod_{m=1}^M P(q_m, \mathbf{w}_m).$$

Moreover, it is customary (for simplicity) to separate the structure of the tree (q_m) and the terminal weights ($\mathbf{w}_m$), so that by a Bayesian conditioning

$$P((q_1, \mathbf{w}_1), \dots, (q_M, \mathbf{w}_M), \sigma^2) = \underbrace{P(\sigma^2)}_{\text{noise term}} \prod_{m=1}^M \underbrace{P(\mathbf{w}_m | q_m)}_{\text{tree weights}} \underbrace{P(q_m)}_{\text{tree struct.}} \quad (9.14)$$

It remains to formulate the assumptions for each of the three parts.

We start with the trees' structures, q_m . Trees are defined by their splits (at nodes) and these splits are characterized by the splitting variable and the splitting level. First, the size of trees is parametrized such that a node at depth d is nonterminal with probability given by

$$\alpha(1 + d)^{-\beta}, \quad \alpha \in (0, 1), \quad \beta > 0. \quad (9.15)$$

The authors recommend to set $\alpha = 0.95$ and $\beta = 2$. This gives a probability of 5% to have 1 node, 55% to have 2 nodes, 28% to have 3 nodes, 9% to have 4 nodes and 3% to have 5 nodes. Thus, the aim is to force relatively shallow structures.

Second, the choice of splitting variables is driven by a generalized Bernoulli (categorical) distribution which defines the odds of picking one particular feature. In the original paper by Chipman, George, and McCulloch (2010), the vector of probabilities was uniform (each predictor has the same odds of being chosen for the split). This vector can also be random and sampled from a more flexible Dirichlet distribution. The level of the split is drawn uniformly on the set of possible values for the chosen predictor.

Having determined the prior of structure of the tree q_m , it remains to fix the terminal values at the leaves ($\mathbf{w}_m|q_m$). The weights at all leaves are assumed to follow a Gaussian distribution $\mathcal{N}(\mu_\mu, \sigma_\mu^2)$, where $\mu_\mu = (y_{\min} + y_{\max})/2$ is the center of the range of the label values. The variance σ_μ^2 is chosen such that μ_μ plus or minus two times σ_μ^2 covers 95% of the range observed in the training dataset. Those are default values and can be altered by the user.

Lastly, for computational purposes similar to those of linear regressions, the parameter σ^2 (the variance of ϵ in (9.12)) is assumed to follow an inverse Gamma law $\text{IG}(\nu/2, \lambda\nu/2)$ akin to that used in Bayesian regressions. The parameters are by default computed from the data so that the distribution of σ^2 is realistic and prevents overfitting. We refer to the original article, section 2.2.4, for more details on this topic.

In sum, in addition to M (number of trees), the prior depends on a small number of parameters: α and β (for the tree structure), μ_μ and σ_μ^2 (for the tree weights) and ν and λ (for the noise term).

9.5.3 Sampling and predictions

The posterior distribution in (9.13) cannot be obtained analytically but simulations are an efficient shortcut to the model (9.12). Just as in Gibbs and Metropolis-Hastings sampling, the distribution of simulations is expected to converge to the sought posterior. After some burn-in sample, a prediction for a newly observed set $\mathbf{x}_*$ will simply be the average (or median) of the predictions from the simulations. If we assume S simulations after burn-in, then the average is equal to

$$\tilde{y}(\mathbf{x}_*) := \frac{1}{S} \sum_{s=1}^S \sum_{m=1}^M \mathcal{T}_m \left(q_m^{(s)}, \mathbf{w}_m^{(s)}, \mathbf{x}_* \right).$$

The complex part is naturally to generate the simulations. Each tree is sampled using the Metropolis-Hastings method: a tree is proposed, but it replaces the existing one only under some (possibly random) criterion. This procedure is then repeated in a Gibbs-like fashion.

Let us start with the MH building block. We seek to simulate the conditional distribution

$$(q_m, \mathbf{w}_m) \mid (q_{-m}, \mathbf{w}_{-m}, \sigma^2, \mathbf{y}, \mathbf{x}),$$

where q_{-m} and $\mathbf{w}_{-m}$ collect the structures and weights of all trees except for tree number m . One tour de force in BART is to simplify the above Gibbs draws to

$$(q_m, \mathbf{w}_m) \mid (\mathbf{R}_m, \sigma^2),$$

where $\mathbf{R}_m = \mathbf{y} - \sum_{l \neq m} \mathcal{T}_l(q_l, \mathbf{w}_l, \mathbf{x})$ is the partial residual on a prediction that excludes the m^{th} tree.

The new MH proposition for q_m is based on the previous tree and there are three possible (and random) alterations to the tree:

- growing a terminal node (increase the complexity of the tree by adding a supplementary leaf);
- pruning a pair of terminal nodes (the opposite operation: reducing complexity);
- changing splitting rules.

For simplicity, the third option is often excluded. Once the tree structure is defined (i.e., sampled), the terminal weights are independently drawn according to a Gaussian distribution $\mathcal{N}(\mu_\mu, \sigma_\mu^2)$.

After the tree is sampled, the MH principle requires that it be accepted or rejected based on some probability. This probability increases with the odds that the new tree increases the likelihood of the model. Its detailed computation is cumbersome and we refer to section 2.2 in Sparapani, Spanbauer, and McCulloch (2019) for details on the matter.

Now, we must outline the overarching Gibbs procedure. First, the algorithm starts with trees that are simple nodes. Then, a specified number of loops include the following *sequential* steps:

Step	Task
1	sample $(q_1, \mathbf{w}_1) \mid (\mathbf{R}_1, \sigma^2)$;
2	sample $(q_2, \mathbf{w}_2) \mid (\mathbf{R}_2, \sigma^2)$;
...	...;
m	sample $(q_m, \mathbf{w}_m) \mid (\mathbf{R}_m, \sigma^2)$;
...	...;
M	sample $(q_M, \mathbf{w}_M) \mid (\mathbf{R}_M, \sigma^2)$; (last tree)

Step	Task
M+1	sample σ^2 given the full residual $\mathbf{R} = \mathbf{y} - \sum_{l=1}^M \mathcal{T}_l(q_l, \mathbf{w}_l, \mathbf{x})$

At each step m , the residual $\mathbf{R}_m$ is updated with the values from step $m - 1$. We illustrate this process in Figure 9.3 in which $M = 3$. At step 1, a partition is proposed for the first tree, which is a simple node. In this particular case, the tree is accepted. In this scheme, the terminal weights are omitted for simplicity. At step 2, another partition is proposed for the tree, but it is rejected. In the third step, the proposition for the third is accepted. After the third step, a new value for σ^2 is drawn and a new round of Gibbs sampling can commence.

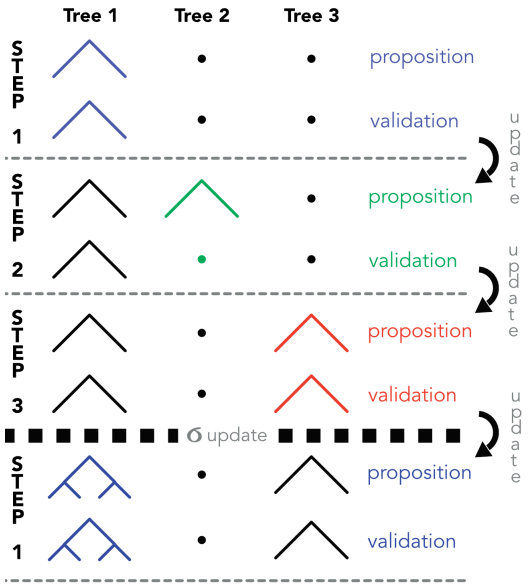


FIGURE 9.3: Diagram of the MH/Gibbs sampling of BARTs. At step 2, the proposed tree is not validated.

9.5.4 Code

There are several R packages that implement BART methods: *BART*, *bartMachine* and an older one (the original), *BayesTree*. The first one is highly efficient, hence we work with it. We resort to only a few parameters, like the power and base, which are the β and α defined in (9.15). The program is a bit verbose and delivers a few parametric details.

```

library(BART)
fit_bart <- gbart(
  x.train = dplyr::select(training_sample, features_short) %>%
    data.frame(),
  y.train = dplyr::select(training_sample, R1M_Usd) %>%
    as.matrix() ,
  x.test = dplyr::select(testing_sample, features_short) %>%
    data.frame(),
  type = "wbart",
  ntree = 20,
  nskip = 100,
  ndpost = 200,
  power = 2,
  base = 0.95)

```

Load package
 # Main function
 # Training features
 # Training label
 # Testing features
 # Option: label is continuous
 # Number of trees in the model
 # Size of burn-in sample
 # Number of posteriors drawn
 # beta in the tree structure prior
 # alpha in the tree structure prior

```
## *****Calling gbart: type=1
## *****Data:
## data:n,p,np: 198128, 7, 70208
## y1,yn: -0.049921, 0.024079
## x1,x[n*p]: 0.010000, 0.810000
## xp1,xp[np*p]: 0.270000, 0.880000
## *****Number of Trees: 20
## *****Number of Cut Points: 100 ... 100
## *****burn,nd,thin: 100,200,1
## *****Prior:beta,alpha,tau,nu,lambda,offset: 2,0.95,1.57391,3,2.84908e-31,0.0139209
## *****sigma: 0.000000
## *****w (weights): 1.000000 ... 1.000000
## *****Dirichlet:sparse,theta,omega,a,b,rho,augment: 0,0,1,0.5,1,7,0
## *****printevery: 100
##
## MCMC
## done 0 (out of 300)
## done 100 (out of 300)
## done 200 (out of 300)
## time: 31s
## trcnt,tecnt: 200,200
```

Once the model is trained,²⁰ we evaluated its performance. We simply compute the hit ratio. The predictions are embedded within the fit variable, under the name ‘*yhat.test*’.

```
mean(fit_bart$yhat.test * testing_sample$R1M_Usd > 0)
```

```
## [1] 0.5432794
```

The performance *seems* reasonable but is by no means impressive. The data from all sampled trees is available in the *fit_bart* variable. It has nonetheless a complex structure (as is often the case with trees). The simplest information we can extract is the value of σ across all 300 simulations (see Figure 9.4).

```
data.frame(simulation = 1:300, sigma = fit_bart$sigma) %>%
  ggplot(aes(x = simulation, y = sigma)) + geom_point(size = 0.7)
```

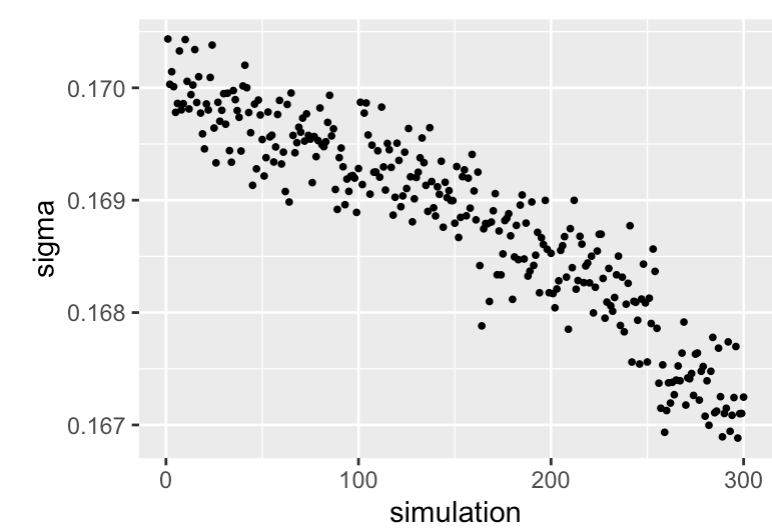


FIGURE 9.4: Evolution of sigma across BART simulations.

And we see that, as the number of samples increases, σ decreases.

```
.container-fluid main { max-width: 60rem; }
```

